



A CAPSTONE PROJECT BRIEF FOR

General Assembly Bahrain

Software Engineering Bootcamp · Capstone Sprint

RAL HR: a modular HR system, built in pieces

Prepared by RAL Technologies

August 2026

We're building HR software for Bahrain. We would love your students to build a piece of it with us.

RAL Technologies is a Bahraini software company, and one of the things we are building is RAL HR, an HR and payroll platform made specifically for Bahrain, because the global tools do not really understand how work here happens.

What we would like your students to build is the ground up version of it. Not a fork of our codebase, not a template with the labels swapped, and not a copy of something already finished. The real thing, designed and written from an empty repository, by them.

The part they will own is the human half of an HR system. Who works here, who is off today, whose work permit expires next month, and who approved what. That half is what every business needs, and it is the part most companies in Bahrain still run on a spreadsheet and a WhatsApp group. Nothing in this brief was invented for a classroom: every requirement in it comes from what a Bahraini business actually asks an HR system to do.

WHY WE THINK THIS IS A GOOD STUDENT PROJECT

- It is real and it is current. Every feature in this brief is on our list because an actual HR manager needs it.
- It splits cleanly. HR software is naturally modular, so several teams can each own a piece without stepping on each other or waiting on each other.
- It maps to your syllabus almost line for line: authentication and authorization, roles, CRUD, RESTful routes, data relationships, an ERD, and a deployed app.
- Nothing sensitive changes hands. We supply fake seed data, so no real employee information ever leaves our systems.
- There is a finish line a beginner can actually reach: one working module, deployed, with a README and a five minute demo they are proud of.

HOW IT IS SHAPED

One small core. Many modules. Always an HR system.

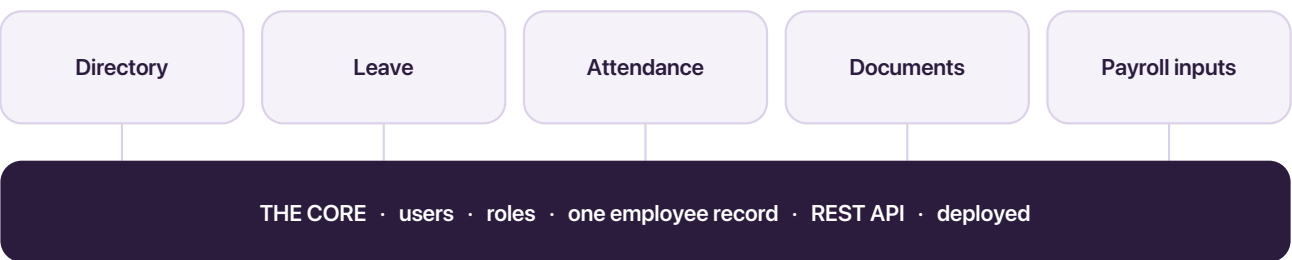
This is the only structural idea in the brief, and everything else follows from it. Every team builds the same small Core: users, roles, and one employee record. On top of that Core, each team picks one module. Leave. Attendance. Documents. Payroll inputs.

Whichever modules they pick, and in whatever order those modules land, the result is still a working HR system. Remove a module and the system gets smaller, not broken. Add one and it gets more useful. That property is not luck, it is how good business software is designed, and it is a lesson worth more than any single feature.

THE ARRANGEMENT

- The Core is not optional and it is identical for everyone: sign in, three roles, one employee record, full CRUD, a REST API, deployed. Roughly the first week.
- Each team then owns exactly one module, front to back. Their own routes, their own screens, their own data, their own tests.
- Modules talk to the Core, not to each other. That is the rule that stops one team's rough night from breaking another team's demo.
- Teams choose from the MVP list (page 5). Any team that finishes early can reach into the nice to have list (page 6) rather than inventing scope.
- On demo day the modules stack into one system, so every student can point at the part that is theirs and still show the whole thing working.

THE PICTURE



Take any of the boxes on top away, or swap their order, and what is left is still an HR system. That is the whole design.

The part everybody builds.

If a student finishes only this page, they have still built something real and it still reads as an HR system. Every item here is already in the syllabus, so no team starts from a cold stop.

Sign in and sign out.

Email and password, hashed properly, with token based auth. Nobody sees a single record before logging in.

Three roles, meaning the same thing everywhere.

Employee sees only their own record. Manager sees their team. HR Admin sees everyone. Enforced on the server, not just hidden in the interface.

One employee record.

Name, job title, department, manager, start date, status. Every other module points at this record by its ID, never by the person's name.

Full CRUD over a REST API.

Create, read, update, and deactivate. Deactivate rather than delete, because HR history is not something a business gets to throw away.

Seeded and deployed.

A seed script that fills the database with about twenty fake employees, and a live URL a stranger can open without a walkthrough.

BEFORE ANY CODE

An ERD, a short set of user stories, and rough wireframes, exactly as their unit projects are already planned. We are happy to review these in office hours before anyone starts building.

The five we would like built first.

One team, one module, two to three weeks. Each is written as a slice a beginner can genuinely finish, and each one answers a complaint we have heard from a real HR manager.

1. People Directory and Records.

Search staff, filter by department, open a profile. HR Admin adds a joiner and marks a leaver. This is the Core with a good face on it, so it suits the team that wants to nail the fundamentals properly.

Why it matters One place where "who works here" is actually true, instead of three spreadsheets that disagree.

2. Leave and Time Off.

Request leave, manager approves or rejects, the balance goes down, and a calendar shows who is out this week.

Why it matters The most common HR request in the country, currently living in a WhatsApp thread nobody can search and nobody can audit.

3. Attendance and Timesheets.

Clock in and out, or log hours against a day, then the manager approves the week. Late arrivals and missing days get flagged automatically.

Why it matters Turns "I think he was in on Tuesday" into a record the business can stand behind when someone asks.

4. Documents and Expiry Alerts.

Upload a document, tag it (CPR, passport, work permit, professional licence), set an expiry date, and see everything expiring in the next ninety days.

Why it matters In Bahrain an expired permit is a fine and a stopped employee. This is quietly the most valuable module on the page.

5. Payroll Inputs and Payslip Viewer.

Allowances, deductions, and overtime for the month, plus a read only payslip screen for the employee. The statutory maths stays with us (page 9).

Why it matters HR stops rebuilding the same spreadsheet on the 25th of every month, and stops pasting numbers between files.

If a team finishes early, or wants to show off.

None of these are required and no team should start here. They are on the list because they are genuinely useful, genuinely finishable in a few days, and they make a portfolio piece look considered rather than rushed.

- **Approvals inbox**

One screen where a manager sees every pending request across modules.

- **Org chart**

Who reports to whom, drawn straight from the manager field.

- **Asset register**

Laptops, phones, and cars, and which employee currently has them.

- **Manager dashboard**

Headcount, who is off, what expires soon, and a CSV export.

- **Ask HR**

Search the company policy document and return the paragraph that answers it.

- **Onboarding checklist**

The tasks a new joiner needs done in week one, ticked off.

- **Announcements**

HR posts, everyone reads. Small, satisfying, and surprisingly wanted.

- **Arabic and English**

A language toggle with a properly laid out right to left interface.

- **Audit log viewer**

A readable history of who changed what, and when.

- **Self service on mobile**

The employee's own view of their own data, designed phone first.

Four rules, and the pieces fit together no matter the order.

This is the one part of the brief we would ask the instructional team to hold students to, because it is what turns several separate projects into one system. Almost none of it is extra work. It is mostly agreeing on names before anyone opens an editor.

AGREE ON THESE ON DAY ONE

- One employee schema. We hand over a starter file with the exact field names, and every team uses those names, including the boring ones.
- Reference people by employee ID, never by name. Names get misspelled, change after a wedding, and repeat across a company. IDs do not.
- Every module lives under its own API path, `/api/leave`, `/api/attendance`, and so on, with the same request and response shape everywhere.
- The three roles mean the same thing in every module. An Employee can never read another Employee's row, in any module, through any route.

The habits that make it feel like a product, not a project.

These are the things we genuinely care about inside RAL HR, cut down to fit a two week sprint. A student who picks up even two of them is ahead of a lot of working developers.

One source of truth.

If a fact lives in two places, the two places will disagree by Thursday. Every module reads the employee record. No module keeps its own copy of a name.

Every change leaves a trace.

Who changed it, what it was before, and when. HR decisions get questioned months later, and "the system says so" only helps if the system remembers.

Rules are settings, not code.

A leave allowance of thirty days belongs in a settings row, not typed inside a function. When the rule changes, someone edits data instead of shipping a release.

Money is stored in whole units.

Store files as integers and format for display. Floating point money loses a fraction per row, quietly, and accountants always find it.

Software records, people pay.

The system produces a file or a report, and a human at a bank moves the money. Keeping that boundary clean protects the business and the developer.

Arabic ships with English, not after it.

If a team goes bilingual, both languages land together and the layout flips properly. Retrofitting right to left later is how weekends disappear.

Built with what they already know.

We wrote this brief against your syllabus, not against our wish list. There is nothing here that asks the instructional team to teach something new.

Where it lands in the units.

Any team that prefers relational over document databases is welcome to build its module in Django and PostgreSQL. HR data suits it well, and we do not mind mixed stacks as long as the field names and API shapes match.

Unit 2: Express, MongoDB, auth, CRUD.

This is the Core, close to one for one. Forms, password hashing, RESTful routes, data relationships, and an ERD are all already on the plan.

Unit 3: React and token auth.

This is every module's front end. Lists, forms, validation, and a fetch to their own API. Postman for testing, exactly as taught.

Unit 4: Python, Django, PostgreSQL.

An alternative path for any module. Leave balances and payroll inputs are naturally relational, so this is a good fit rather than a compromise.

Assessment stays the same.

ERD, user stories, wireframes, deployment. The artefacts you already grade are the artefacts we would ask for.

NOT IN SCOPE, DELIBERATELY

- Statutory payroll maths: SIO contributions, end of service, and WPS salary files. That work cites legal sources and stays with RAL.
- Real employee data. Fake seed data only, supplied by us.
- Anything that moves money. No payment gateways, no bank connections.
- Government or LMRA integrations.
- Any production credential or account of ours.

WHY

Cutting these keeps the sprint finishable, keeps students out of regulated territory, and keeps the privacy question off the table entirely.

NEXT STEPS

Over to you.

That is the whole brief, and none of it is fixed. If a module is too large, too small, or lands in the wrong week for your teaching plan, say so and we will reshape it. We would rather adjust now than hand your students something that does not fit the course.

1

Have a read with the instructional team and tell us what you would change. Module sizes, the order they land in, how the teams are split, the depth of each slice, all of it is adjustable and none of it is precious to us.

2

Once you are happy with the shape, we send the starter pack: the employee schema, fake seed data, a one page glossary of HR terms, and the module list written up as tickets, so students open day one with something to build rather than a blank repository.

3

We will gladly make ourselves available for the kickoff on Sunday 23 August, at your offices at BIBF, at ours, or online, whichever is easiest for you and the students. We are just as happy to stay close through the sprint with office hours, a channel for quick questions, and a walkthrough of how we think about the problem. Tell us how involved you would like us to be and we will work to that.

ANY QUESTIONS, DIRECTLY TO

Hisham Ghareeb, General Manager, RAL Technologies

hisham@raltech.dev · +973 3821 8181 · raltech.dev